# *Arquitetura de Computadores I: Memória*

**Prof. Alexandro Magno dos Santos Adário**

# *Introdução às Memórias*

- **Todo programa executado e todo dado a ser processado, armazenado ou apresentado passa pela memória.**
- **Na prática possui mais impacto no desempenho do sistema que a velocidade do processador**
  - **Muito mais lenta que o processador**
  - **Evita acessos desnecessários aos discos**
- **Tendência de crescimento:**
  - **4 vezes em 3 anos (59% ao ano)**

# *Random Access Memory*

- **RAM ou memória principal é a memória básica do sistema, onde ficam os dados e as instruções para processamento.**
- **Características de construção:**
  - **Volatilidade**
  - **Formada por células**
  - **Acesso direto**
- **Vários tipos, distintas em velocidade (e custo, por conseqüência)**
- **Basicamente há dois tipos:**
  - **SRAM (estática) – não necessita atualização dos dados**
  - **DRAM (dinâmica) – dados atualizados periodicamente**

# Tipos de RAM

| Tipo | Velocidade | Características |
|---|---|---|
| Fast Page Mode DRAM | 28,5 MHz | A leitura de uma coluna é feita com apenas 1 solicitação |
| Extended Data Out DRAM | 40 MHz | A leitura do 2º dado ocorre imediatamente à do 1º |
| Burst EDO DRAM | 66 MHz | O 2º dado começa a ser lido antes do término da leitura do 1º |
| Synchronous DRAM | 125 MHz | Sincroniza com o processador, evitando atrasos |
| Double Data Rate SDRAM | 200 MHz | Dois dados são transmitidos por vez para o processador |
| Synchronous Link DRAM | 400 MHz | Mudança na tecnologia permite que transmita mais por ciclo |
| Rambus DRAM | 600 MHz | Os canais de comunicação são mais largos (mais dados) |
| Direct RDRAM | 800 MHz | Evolução da Rambus, canais ainda mais largos. |

# Memórias Dedicadas

- Memórias acopladas a um dispositivo de E/S
  - Também chamadas de memórias locais
- Buffer de impressora
  - O computador não aguardando impressora
  - Tarefa armazenada até a impressora estar pronta.
- Memória de Vídeo (Video Ram, VRAM)
  - Acessada por dois dispositivos ao mesmo tempo
    - Computador grava dados da imagem
    - E coprocessador da aceleradora lê/grava imagem
  - Mais rápida que a RAM comum
    - WRAM (Windows RAM)
    - SGRAM (Synchronous Graphic RAM)
    - MDRAM (Multibank DRAM)

# *Read-Only Memory*

- Memória apenas de escrita
  - Usuário não pode sobrescrevê-la em operação normal
  - Fixa de fábrica ou através de hardware/software especiais
- Usada para funções básicas de sistemas.
  - Rotinas de mais baixo nível do hardware
  - BIOS (Basic Input/Output System)
  - Teste do sistema
  - Rotina de boot do sistema
  - Controle básico de teclado, monitor e discos
- Vários dispositivos de E/S utilizam ROM para armazenar suas subrotinas

# *Tipos de ROM*

- **ROM**
  - **Vem pronta de fábrica**
  - **Não pode ser nunca reescrita**
- **PROM (*Programmable* ROM)**
  - **Compra-se um chip “em branco”**
  - **Usuário faz sua programação (APENAS 1 VEZ)**
- **EPROM (*Erasable* PROM)**
  - **É uma PROM, mas pode ser apagada**
  - **Utiliza-se luz ultra-violeta para apagar o conteúdo**
- **EEPROM (*Electrically Erasable* PROM)**
  - **Também apagável, mas usando eletricidade**

# *Arquitetura de Computadores I: Memória Cache*

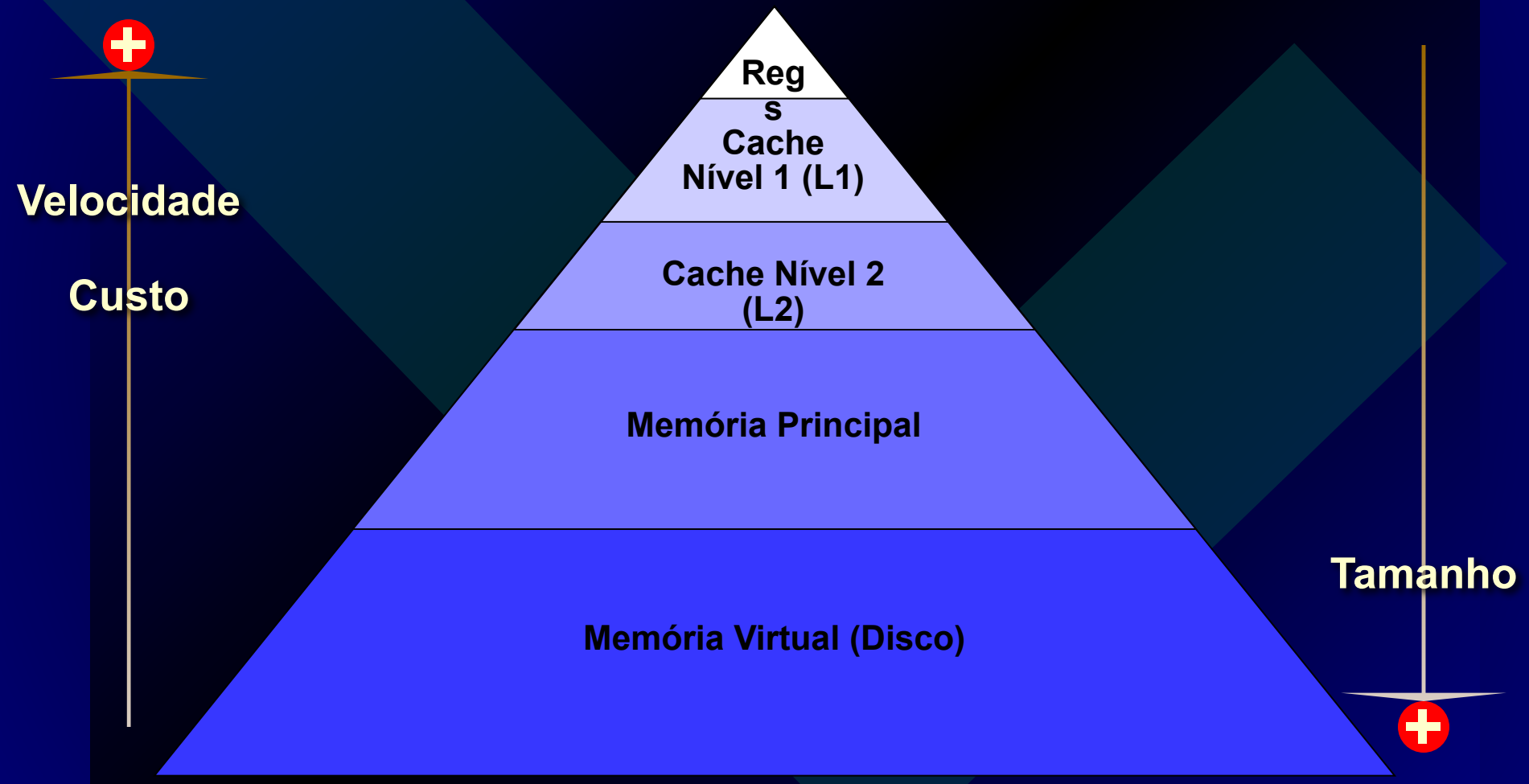
**Prof. Alexandro Magno dos Santos Adário**

**URI – Campus de Erechim**

# *Princípio da Localidade*

- Programadores desejam memória rápida e ilimitada
- O projeto desta memória é inviável (se não impossível)
  - Ilusão para o programador
  - Acesso a uma memória muito pequena embora veloz
  - Um programa não utiliza todo seu espaço de endereçamento de uma vez
- Localidade Temporal
  - Se um endereço é referenciado, tende a ser novamente novamente num tempo próximo
- Localidade Espacial
  - Se um endereço é referenciado, os endereços vizinhos a ele tendem a ser referenciados num tempo próximo.

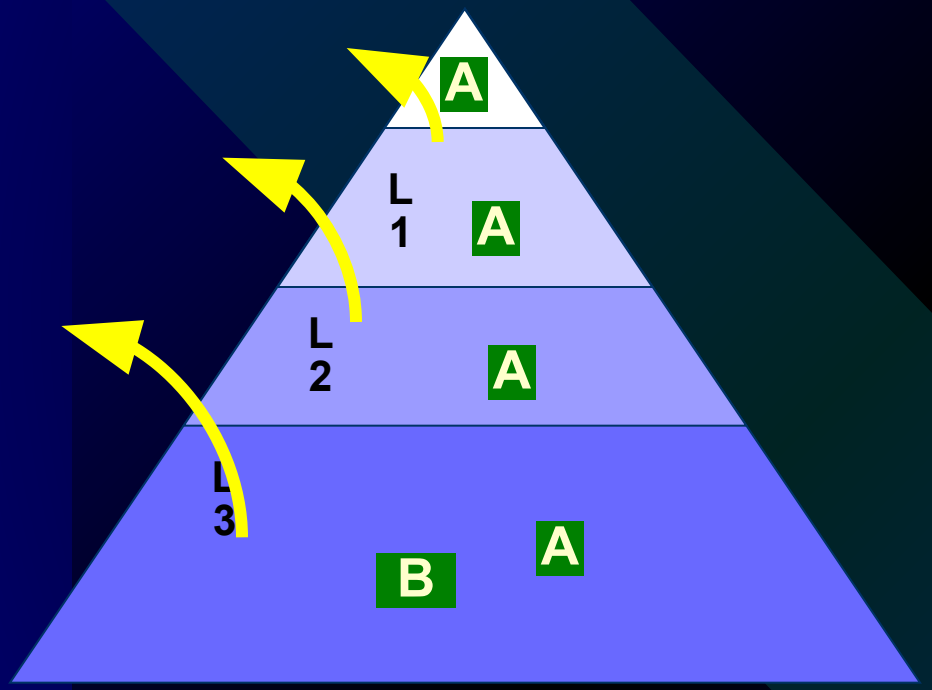
# Hierarquia de Memória



# *Memória Cache*

- O processador é bem mais rápido que a memória
  - Muitas vezes fica aguardando dados/instruções para processar
- A Cache é uma memória de alta velocidade
  - Antecipa a busca de dados que serão acessados
  - Fisicamente mais próxima do processador
- Princípios de Localidade
  - Espacial: proximidade dos dados acessados
  - Temporal: uso freqüente dos mesmos dados
- Divisão em vários níveis para reduzir impacto

# Princípios da Cache




- Um dado é sempre movido um nível por vez
- Os dados existentes nos níveis mais altos são um subconjunto dos dados nos mais baixos
- Os dados dos níveis mais altos sempre estão presentes nos mais baixos.
- Quando o processador solicita um dado e ele está no nível mais alto, ocorre um hit (acerto; oposto: miss).
- Taxa de acerto: fração dos acesos à memória que resultaram em hits.

# *Métodos de Escrita em Cache*

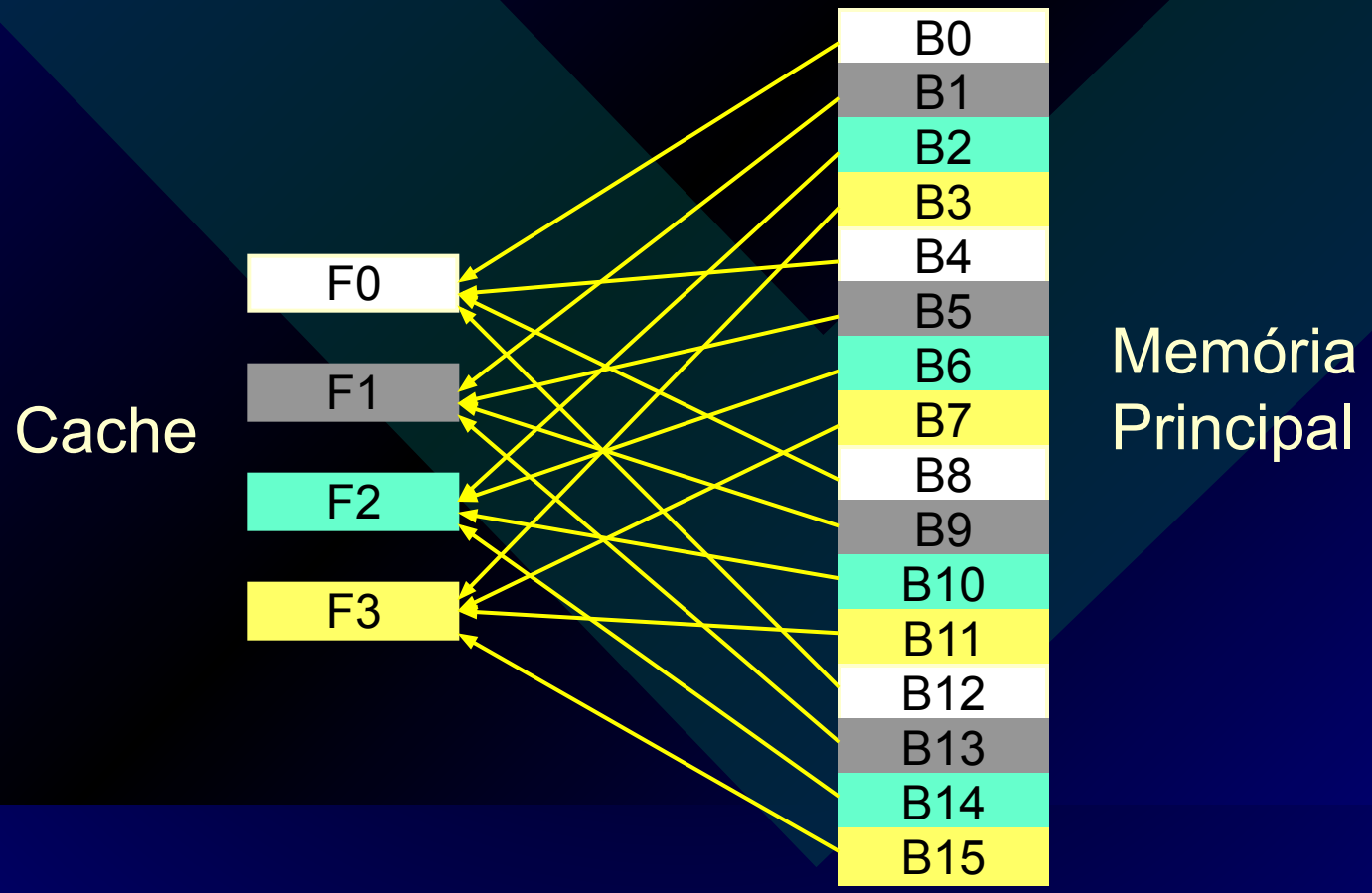
- **Write-through**
  - **Neste método de escrita, os dados são gravados na cache e, imediatamente, na memória principal**
  - **Existe um tempo extra adicional**
- **Write-back**
  - **Os dados só são atualizados na memória principal quando sua posição na cache for necessária para outro dado**
  - **Método mais barato e rápido, mas pode haver inconsistências.**

# Organização da Cache

- Memória principal é dividida em N blocos ($n = 2^S$)
- Cada bloco contém b palavras ($b = 2^W$)
- Tamanho total da memória: $n \times b = 2^S \times 2^W = 2^{S+W}$
- A cache é dividida em <u>m</u> block frames ($m = 2^T$)
- Tamanho total da cache: $m \times b = 2^T \times 2^W = 2^{T+W}$
- Relação de tamanho entre memória principal e cache:
  - N >> M (N é muito maior que M)
- Modos de associação
  - Indicam como um block frame é associado aos blocos
  - Mapeamento direto, Associação completa e Associação por conjunto

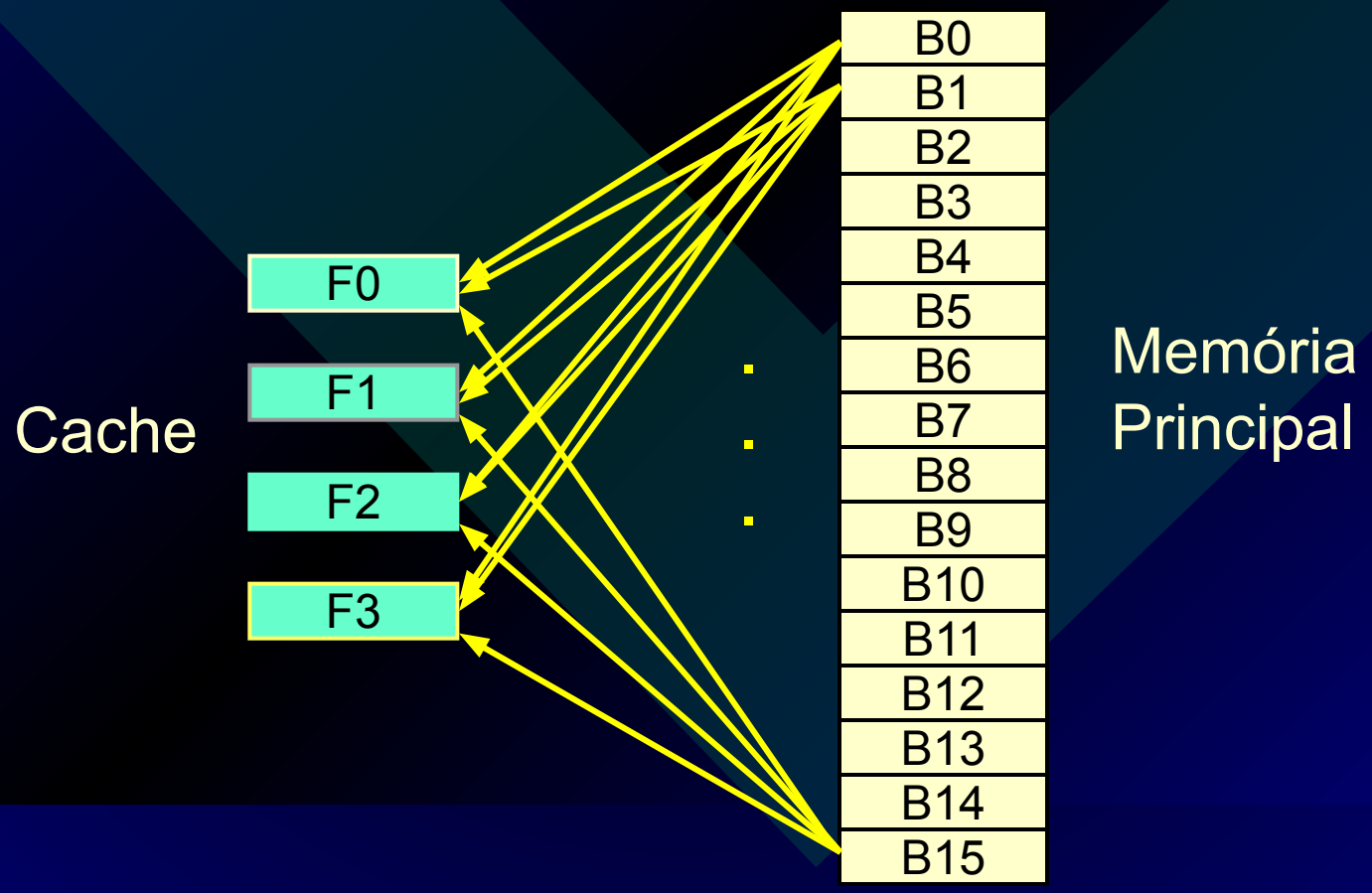
# Mapeamento Direto

- **Um bloco da memória principal só pode ser movido para um e somente um *block frame* da cache.**
- **Um block frame pode receber vários blocos**



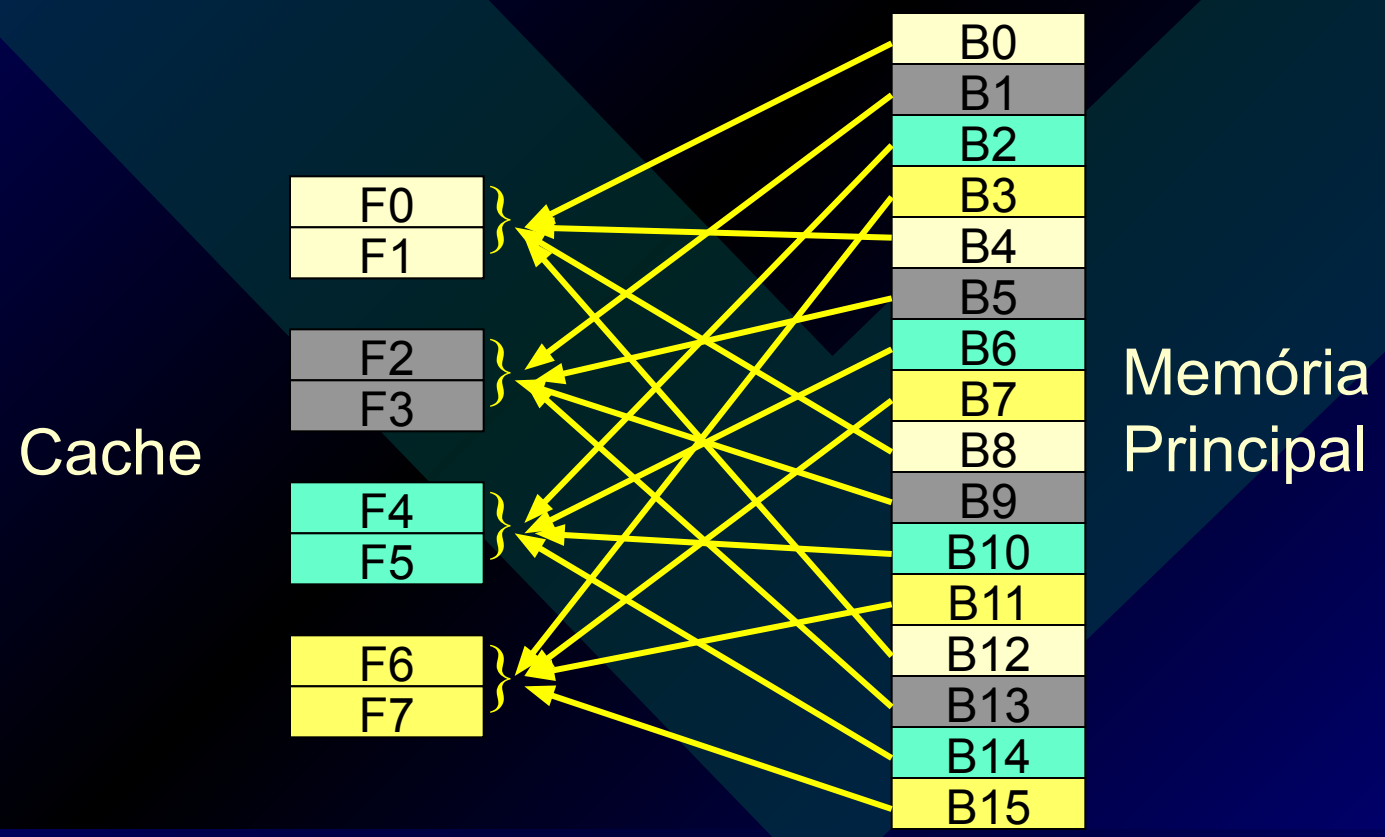
# Associação Completa

- **Um bloco da memória principal pode ser movido para qualquer *block frame* da cache.**

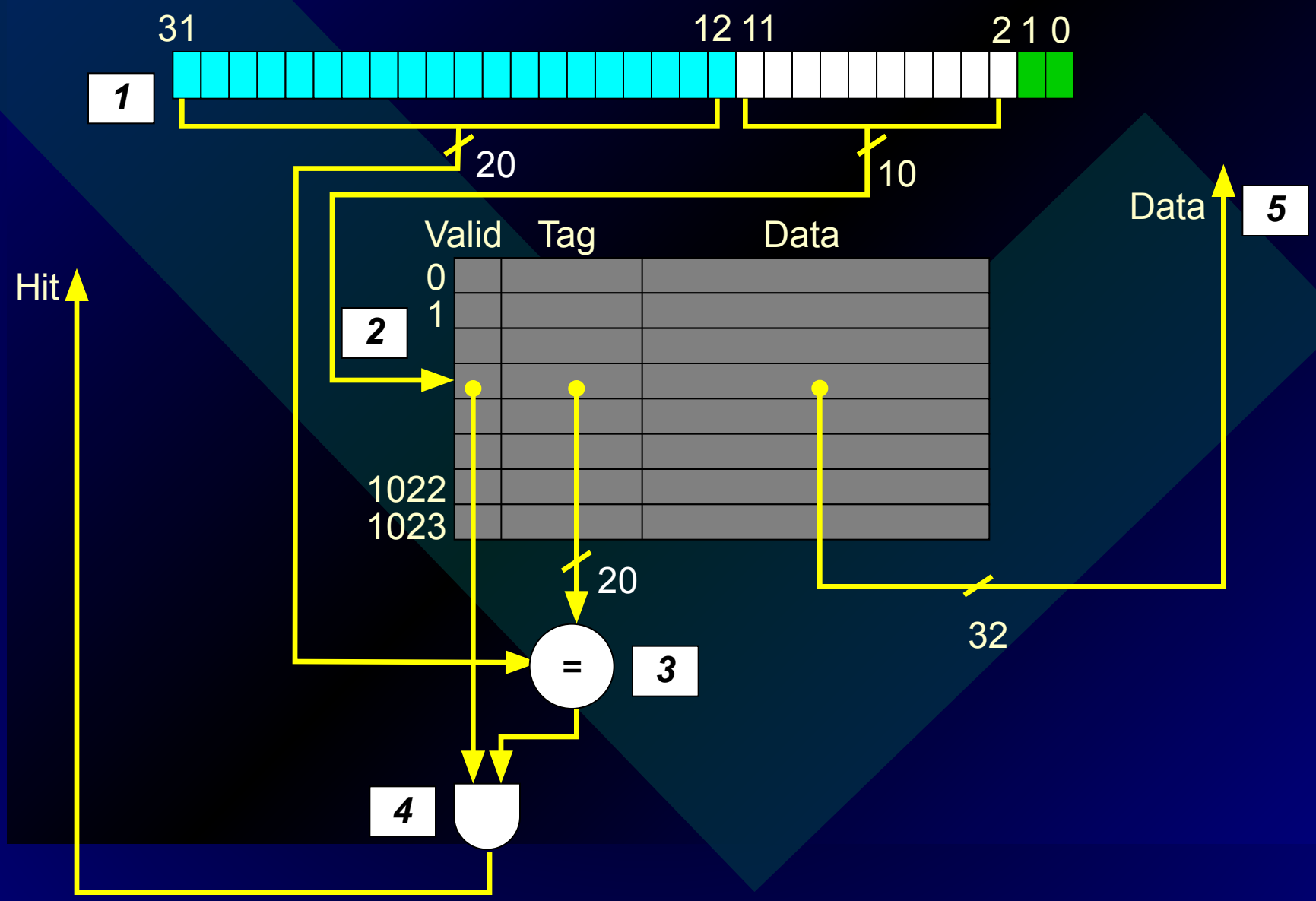


# Associação por Conjunto

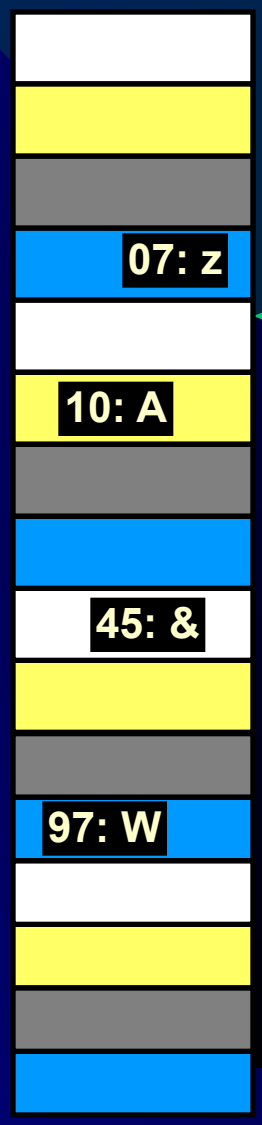
- **Um bloco da memória só pode estar num conjunto de *block frames* da cache, mas dentro do conjunto, pode estare em qualquer um.**



# Estrutura da Cache

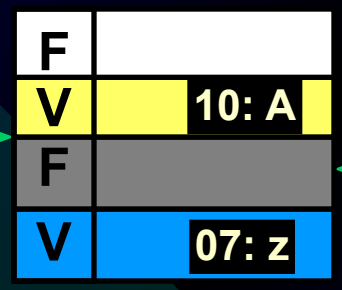


# Funcionamento da Cache




A memória é logicamente dividida em blocos (com vários bytes)

A cache possui blocos do mesmo tamanho da memória



1. O processador solicita leitura de um endereço.
2. O endereço solicitado é comparado com o do bloco correspondente.
3. Se o endereço for o mesmo, e o bit validade for verdadeiro, ocorreu hit e o dado da cache passa ao processador.
4. Senão, a cache irá buscar o bloco (onde está o endereço) na memória e então passar o conteúdo ao processador.

# Exemplo de Acesso à Cache (I)

**Acessos**

| 10110 | 11010 | 10110 | 11010 |
|---|---|---|---|

**Cache**

| Índice | Valid | Tag | | Data | |
|---|---|---|---|---|---|
| 000 | F | | | | |
| 001 | F | 10000 | 00100 | 10000 | 10010 |
| 010 | F | | | | |
| 011 | F | | | | |
| 100 | F | | | | |
| 101 | F | | | | |
| 110 | F | | | | |
| 111 | F | | | | |

# Exemplo de Acesso à Cache (II)

**Acessos**

| | | | |
|---|---|---|---|
| 10110 | 11010 | 10110 | 11010 |
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | F | | |
| 001 | F | | |
| 010 | F | | |
| 011 | F | | |
| 100 | F | | |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (III)

**Acessos**

| 10110 | 11010 | 10110 | 11010 |
|---|---|---|---|
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | F | | |
| 001 | F | | |
| 010 | V | 11 | Mem(11010) |
| 011 | F | | |
| 100 | F | | |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (IV)

**Acessos**

| | | | |
|---|---|---|---|
| 10110 | 11010 | 10110 | 11010 |
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | F | | |
| 001 | F | | |
| 010 | V | 11 | Mem(11010) |
| 011 | F | | |
| 100 | F | | |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (V)

**Acessos**

| 10110 | 11010 | 10110 | 11010 |
|---|---|---|---|
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | F | | |
| 001 | F | | |
| 010 | V | 11 | Mem(11010) |
| 011 | F | | |
| 100 | F | | |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (VI)

**Acessos**

| | | | |
|---|---|---|---|
| 10110 | 11010 | 10110 | 11010 |
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | V | 10 | Mem(10000) |
| 001 | F | | |
| 010 | V | 11 | Mem(11010) |
| 011 | F | | |
| 100 | F | | |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (VII)

**Acessos**

| | | | |
|---|---|---|---|
| 10110 | 11010 | 10110 | 11010 |
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | V | 10 | Mem(10000) |
| 001 | F | | |
| 010 | V | 11 | Mem(11010) |
| 011 | F | | |
| 100 | V | 00 | Mem(00100) |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (VIII)

**Acessos**

| | | | |
|---|---|---|---|
| 10110 | 11010 | 10110 | 11010 |
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | V | 10 | Mem(10000) |
| 001 | F | | |
| 010 | V | 11 | Mem(11010) |
| 011 | F | | |
| 100 | V | 00 | Mem(00100) |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# Exemplo de Acesso à Cache (IX)

**Acessos**

| | | | |
|---|---|---|---|
| 10110 | 11010 | 10110 | 11010 |
| 10000 | 00100 | 10000 | 10010 |

**Cache**

| Índice | Valid | Tag | Data |
|---|---|---|---|
| 000 | V | 10 | Mem(10000) |
| 001 | F | | |
| 010 | V | 10 | Mem(10010) |
| 011 | F | | |
| 100 | V | 00 | Mem(00100) |
| 101 | F | | |
| 110 | V | 10 | Mem(10110) |
| 111 | F | | |

# *Arquitetura de Computadores I: Memória Virtual*

**Prof. Alexandro Magno dos Santos Adário**

**URI – Campus de Erechim**

# Objetivos da Memória Virtual

- Permitir o compartilhamento do espaço de memória
  - Diferentes programas do mesmo usuário
  - Programas de diferentes usuários
- Superar a limitação física da memória principal
  - Possibilitar execução de programas maiores
  - Explorar o conceito de localidade de forma eficiente

**Memória Cache** ⟷ **Memória Principal**

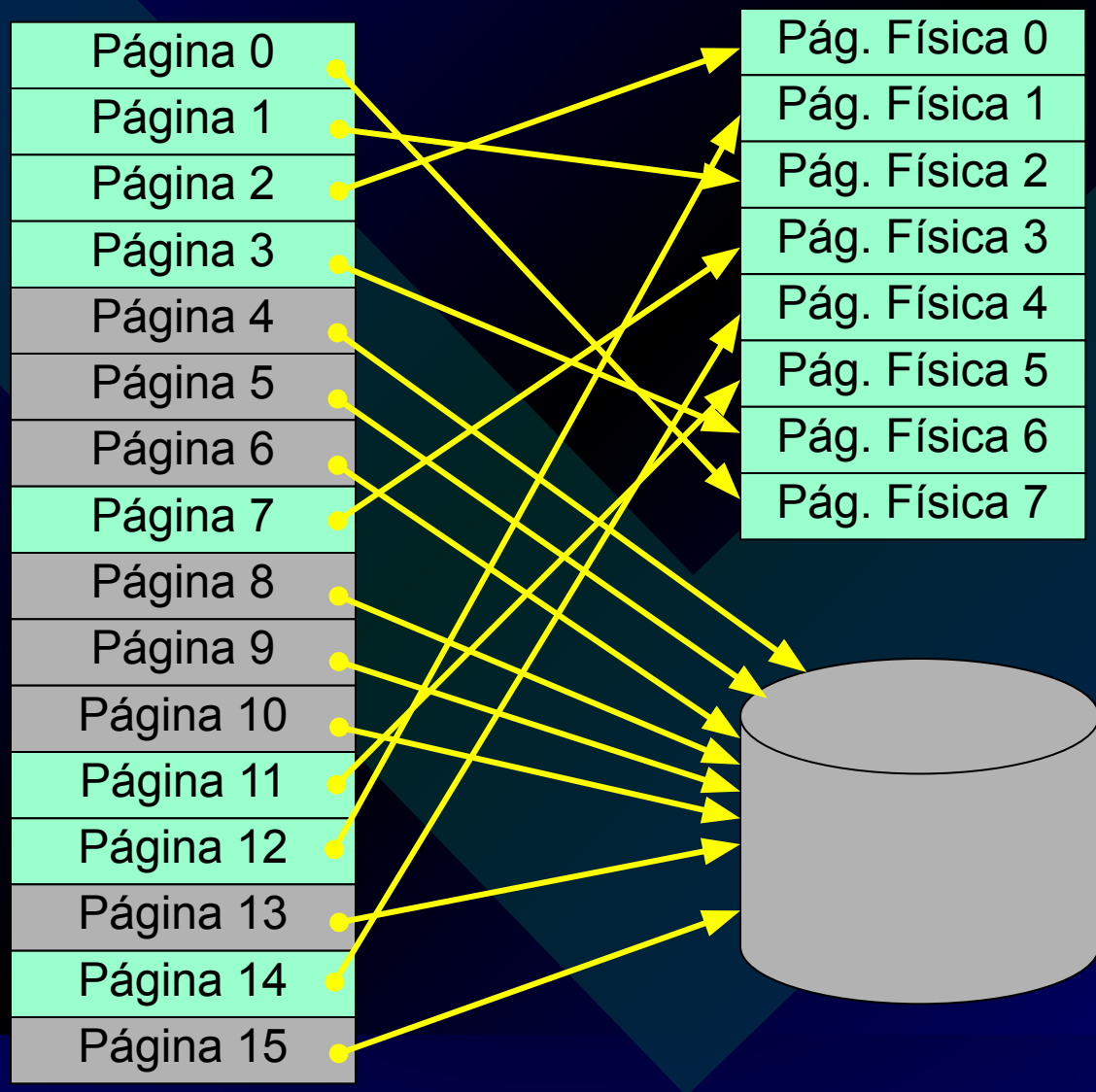
***Comportamento Similar***

**Memória Principal** ⟷ **Memória Secundária**

# Definições

- Espaço de Endereçamento
  - Toda memória que pode ser endereçada pelo processador
- Unidade de Transferência
  - Memória principal : página física (physical page)
  - Memória secundária : página (page)
- Página não encontrada : page fault (falta de página)
- Mapeamento de Endereço:
  - Processador produz endereços virtuais
  - Tais endereços são convertidos para endereços físicos

# Mapeamento de Páginas



# Tabela de Alocação de Páginas

- Para determinar a localização da página
  - Indica ocorrência de page fault (página na memória virtual)
- Estrutura da Tabela de Páginas (TLB)
  - Translation-Lookaside Buffer
  - Índice: entrada para o endereço virtual
  - Bit de Residência: Indica se é página física ou não
  - Endereço Físico: informa a localização da página física

| Índice | Residente | Endereço Físico |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

# Exemplo de TLB

- Computador com endereços de 32 bits, 16 MB de memória e páginas de 64 kB
  - 32 bits => Espaço de endereçamento = 4 GB
  - Páginas de 64 kB => 16 bits para endereçar cada byte
  - Índice = 32 bits (Total) - 16 bits (página) = 16 bits
  - 16 MB de memória => 24 bits de endereço
  - Endereço de páginas físicas = 24 –16 = 8 bits (256 pág.)

| Índice | Residente | Endereço Físico |
|---|---|---|
| 16 bits | 1 bit | 8 bits |
| | | |
| | | |
| | | |

# *Esquema de Acesso a TLB*

- **Processador fornece um endereço (virtual)**
- **A página virtual do endereço é localizada na TLB**
- **Se a página está residente, termina a busca e o endereço físico é acessado na memória física**
- **Se a página não está residente, o controle é passado ao sistema operacional**
- **Sistema operacional localiza a página na memória secundária e decide onde mapeá-la na primária**

# *Paginação de Memória*

- **O bom desempenho de um sistema de memória virtual está embasado na política de substituição das páginas**
- **Paginação é tarefa do Sistema Operacional, mas é um processo altamente integrado ao hardware**
- **Há pelo menos 3 métodos bastante utilizados:**
  - **LRU (*Least Recently Used*): página cujo último acesso é o mais antigo de todas é substituída**
  - **FIFO (*First In First Out*): página mais antiga na memória é substituída**
  - **LFU (*Least Frequently Used*): página com menor número de acessos**

# Exemplo 1 (Paginação LRU)

- **Páginas 10, 9, 11, 12, 1, 3, 4, 10, 12, 11, 4, 5, 6, 9, 10 e 7**

| PF | PV | UA | PF | PV | UA | PF | PV | UA | PF | PV | UA | PF | PV | UA | PF | PV | UA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | | | 0 | 10 | 0 | 0 | 10 | 7 | 0 | 10 | 7 | 0 | 10 | 14 | 0 | 10 | 14 |
| 1 | | | 1 | 9 | 1 | 1 | 9 | 1 | 1 | 6 | 12 | 1 | 6 | 12 | 1 | 6 | 12 |
| 2 | | | 2 | 11 | 2 | 2 | 11 | 9 | 2 | 11 | 9 | 2 | 11 | 2 | 2 | 11 | 2 |
| 3 | | | 3 | 12 | 3 | 3 | 12 | 8 | 3 | 12 | 8 | 3 | 12 | 8 | 3 | 12 | 8 |
| 4 | | | 4 | 1 | 4 | 4 | 1 | 4 | 4 | 1 | 4 | 4 | 9 | 13 | 4 | 9 | 13 |
| 5 | | | 5 | 3 | 5 | 5 | 3 | 5 | 5 | 3 | 5 | 5 | 3 | 5 | 5 | 7 | 15 |
| 6 | | | 6 | 4 | 6 | 6 | 4 | 10 | 6 | 4 | 10 | 6 | 4 | 10 | 6 | 4 | 10 |
| 7 | | | 7 | | | 7 | 5 | 11 | 7 | 5 | 11 | 7 | 5 | 11 | 7 | 5 | 11 |

PF – Página Física, PV – Página Virtual, UA – Último Acesso

# Exemplo 2 (Paginação FIFO)

- **Páginas 10, 9, 11, 12, 1, 3, 4, 10, 12, 11, 4, 5, 6, 9, 10 e 7**

| PF | PV | M | PF | PV | M | PF | PV | MP | PF | PV | MP | PF | PV | MP | PF | PV | MP |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | | | 0 | 10 | 0 | 0 | 10 | 0 | 0 | 6 | 12 | 0 | 6 | 12 | 0 | 6 | 12 |
| 1 | | | 1 | 9 | 1 | 1 | 9 | 1 | 1 | 9 | 1 | 1 | 10 | 14 | 1 | 10 | 14 |
| 2 | | | 2 | 11 | 2 | 2 | 11 | 2 | 2 | 11 | 2 | 2 | 11 | 2 | 2 | 7 | 15 |
| 3 | | | 3 | 12 | 3 | 3 | 12 | 3 | 3 | 12 | 3 | 3 | 12 | 3 | 3 | 12 | 3 |
| 4 | | | 4 | 1 | 4 | 4 | 1 | 4 | 4 | 1 | 4 | 4 | 1 | 4 | 4 | 1 | 4 |
| 5 | | | 5 | 3 | 5 | 5 | 3 | 5 | 5 | 3 | 5 | 5 | 3 | 5 | 5 | 3 | 5 |
| 6 | | | 6 | 4 | 6 | 6 | 4 | 6 | 6 | 4 | 6 | 6 | 4 | 6 | 6 | 4 | 6 |
| 7 | | | 7 | | | 7 | 5 | 11 | 7 | 5 | 11 | 7 | 5 | 11 | 7 | 5 | 11 |

PF – Página Física, PV – Página Virtual, MP – Momento da paginação

# Exemplo 3 (Paginação LFU)

- **Páginas 10, 9, 11, 12, 1, 3, 4, 10, 12, 11, 4, 5, 6, 9, 10 e 7**

| PF | PV | F/ | PF | PV | F/ | PF | PV | FA | PF | PV | FA | PF | PV | FA | PF | PV | FA |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | | | 0 | 10 | 0 | 0 | 10 | 1 | 0 | 10 | 1 | 0 | 10 | 2 | 0 | 10 | 2 |
| 1 | | | 1 | 9 | 0 | 1 | 9 | 0 | 1 | 9 | 0 | 1 | 9 | 1 | 1 | 9 | 1 |
| 2 | | | 2 | 11 | 0 | 2 | 11 | 1 | 2 | 11 | 1 | 2 | 11 | 1 | 2 | 11 | 1 |
| 3 | | | 3 | 12 | 0 | 3 | 12 | 1 | 3 | 12 | 1 | 3 | 12 | 1 | 3 | 12 | 1 |
| 4 | | | 4 | 1 | 0 | 4 | 1 | 0 | 4 | 6 | 0 | 4 | 6 | 0 | 4 | 6 | 0 |
| 5 | | | 5 | 3 | 0 | 5 | 3 | 0 | 5 | 3 | 0 | 5 | 3 | 0 | 5 | 7 | 0 |
| 6 | | | 6 | 4 | 0 | 6 | 4 | 1 | 6 | 4 | 1 | 6 | 4 | 1 | 6 | 4 | 1 |
| 7 | | | 7 | | | 7 | 5 | 0 | 7 | 5 | 0 | 7 | 5 | 0 | 7 | 5 | 0 |

PF – Página Física, PV – Página Virtual, FA – Freqüência de Acesso